

Exception Handling

Why we can Handle the Exception :-

1. Using Only try block :-

- The try block alone cannot handle an exception.
- It is used only to enclose the code that might cause an exception.
- Without a catch or finally block, using only try will cause a compile-time error.

Example –

```
try {  
  
    int a = 10 / 0;           // ArithmeticException  
  
}  
  
System.out.println("Hello"); //Compile-time error
```

2. Using Only catch block :-

- A catch block cannot be used alone.
- It must always be preceded by a try block.
- If you use only a catch block, it will cause a compile-time error.

Example –

```
catch (Exception e) {  
  
    System.out.println("Exception handled");  
  
}
```

3. Using only finally block :-

- The finally block can be used alone, but it cannot handle exceptions.
- It is used to execute important code whether an exception occurs or not.
- If an exception occurs inside finally and is not handled, the program will still terminate abnormally.

Example –

```
finally {  
  
    System.out.println("Finally block executed"); }  
  
}
```

4. Using try & catch block :-

- This is the most common and valid way to handle exceptions in Java.
- The try block contains risky code that may cause an exception.
- The catch block handles that exception when it occurs.
- The program continues normally after handling the exception.

Example –

```
package exception_handling;

public class str_handling {

    public static void main(String[] args) {
        String str=null;
        try
        {
            System.out.println(str.length());
        }
        catch(Exception e)
        {
            System.out.println(e);
        }

    }

}
```

Output –

[java.lang.NullPointerException](#): Cannot invoke "String.length()" because "str" is null

5. Using try, catch & finally block :-

- This is the complete form of exception handling.
- The try block contains code that may throw an exception.
- The catch block handles the exception.
- The finally block executes always, whether an exception occurs or not, and is usually used to release resources.

Example –

```
package exception_handling;

public class finally_demo {

    public static void main(String[] args) {
        try
        {
            int x=4/0;
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
        finally
        {
            System.out.println("Finally Block Executed");
        }
        System.out.println("Bye");
    }
}
```

Output –

```
java.lang.ArithmeticException: / by zero
Finally Block Executed
Bye
```

6. Using try & finally block :-

- The try block contains code that may throw an exception.
- The finally block always executes, whether an exception occurs or not.
- Using try and finally without a catch is valid, but the exception is not handled — it will propagate to the caller.

Example –

```
package exption_handling;
public class TryFinallyExample {
    public static void main(String[] args) {
        try
        {
            int a =10/0;
            System.out.println("Inside try block");
        }
        finally
        {
            System.out.println("Finally block executed");
        }
        System.out.println("Program continues...");
    }
}
```

Output –

```
Finally block executed
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at exption_handling.TryFinallyExample.main(TryFinallyExample.java:6)
```

7. Using multiple exception in single try block :-

- The try block contains code that may throw more than one exception.
- You can handle them using:
 - Multiple catch blocks (one for each exception type)
 - Single multi-catch block (Java 7+ syntax)

Example –

```
package exception_handling;

public class MultipleExceptionExample {
    public static void main(String[] args) {
        try
        {
            int a=10/0;           // ArithmeticException
            String str=null;
            System.out.println(str.length()); // NullPointerException
        }
        catch(Exception e)
        {
            System.out.println("Exception is handled: "+e);
        }

        System.out.println("Program continues...");
    }
}
```

Output –

[ArithmeticException](#) handled: [java.lang.ArithmeticException](#): / by zero
Program continues...

8. Using nested try-catch block :-

- A nested try-catch is a try block inside another try block.
- It is used when one risky operation depends on another risky operation.
- Allows handling different exceptions at different levels.

Example –

```
package exception_handling;

public class NestedTryCatchExample {
    public static void main(String[] args) {
        try {
            int[] arr={1,2,3};

            try {
                System.out.println(arr[5]);    // ArrayIndexOutOfBoundsException
            }
            catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: " + e);
            }

            int a = 10 / 0;    // ArithmeticException
        }
        catch (ArithmeticException e) {
            System.out.println("Outer catch: " + e);
        }

        System.out.println("Program continues...");
    }
}
```

Output –

Inner catch: [java.lang.ArrayIndexOutOfBoundsException](#): Index 5 out of bounds for length 3
Outer catch: [java.lang.ArithmeticException](#): / by zero
Program continues...

9. Using multiple catch block :-

- A single try block can be followed by more than one catch block.
- Each catch handles a different type of exception.
- Only the first matching catch block is executed.

Example –

```
package exception_handling;

public class handle_exception {

    public static void main(String[] args) {
        String str=null;
        int arr[]= {12,14};
        try{
            int a=5/0;           //ArithmeticException
            System.out.println(str.length()); //Null Pointer Exception
            System.out.println(arr[3]);       //ArrayIndexOutOfBoundsException
        }
        catch(ArithmeticException e) {
            System.out.println(e);
        }
        catch(NullPointerException e1) {
            System.out.println(e1);
        }
        catch(Exception e2) {
            System.out.println(e2);
        }
        System.out.println("Program continues...");
    }
}
```

Output –

```
java.lang.ArithmeticException: / by zero
Program continues...
```

10. Using multiple try-catch block in catch block :-

- Sometimes, while handling an exception, additional code may also throw exceptions.
- In such cases, you can use a try-catch block inside a catch block.
- This ensures that all exceptions are properly handled, even those occurring during exception handling.

Example –

```
package exception_handling;

public class TryCatchInCatchExample {
    public static void main(String[] args) {
        try {
            int a=10/0;    // ArithmeticException
        }
        catch(ArithmeticException e) {
            System.out.println("Outer catch: " + e);

            try {
                String str=null;
                System.out.println(str.length()); // NullPointerException
            }
            catch (NullPointerException ne) {
                System.out.println("Inner catch: " + ne);
            }
        }

        System.out.println("Program continues...");
    }
}
```

Output –

Outer catch: [java.lang.ArithmeticException](#): / by zero
Inner catch: [java.lang.NullPointerException](#): Cannot invoke "String.length()" because "str" is null
Program continues...

Summarizing ways to handle the Exceptions in Table Format :-

Method	Can Handle Exception?	Description
Only try	No	Encloses risky code but needs catch/finally
Only catch	No	Must follow a try block
Only finally	No	Executes always but does not handle exception
try + catch	Yes	Most common way to handle exceptions
try + catch + finally	Yes	Handles exception and executes code always (e.g., resource cleanup)
try + finally	Partially	Executes code always; exception propagates if occurs
Multiple exceptions in single try	Yes	Handle using multiple catch or multi-catch (Java 7+)
Nested try-catch	Yes	Handles exceptions at different levels
Multiple catch blocks	Yes	Each catch handles a different exception
try-catch inside catch	Yes	Handles exceptions occurring during exception handling

Some Important Questions of Multithreading for Mock Interview :-

Q1. What is a Thread?

A thread is the smallest unit of a process that can run independently. It is a lightweight subprocess used for multitasking within a program.

Q2. What is a Process?

A process is an independent program in execution that has its own memory space and resources.

Q3. Difference between Process and Thread:

Process	Thread
Independent program in execution	Smallest part of a process
Has its own memory	Shares memory with other threads
Heavyweight	Lightweight
Slow communication	Fast communication
Independent of other processes	Dependent on parent process
Failure doesn't affect others	Failure may affect other threads

Q4. What is Multitasking & Multiprocessing?

- Multitasking: Executing multiple tasks at the same time (like running multiple apps).
- Multiprocessing: Using multiple CPUs or cores to execute processes simultaneously.

Q5. What is Multithreading? Why is it needed in Java?

Multithreading allows multiple threads to execute concurrently within a single program. It is needed for:

- Better CPU utilization
- Faster execution
- Smooth user experience (no blocking)

Q6. If two threads share resources or access the same data at the same time, what will happen?

It may cause data inconsistency or race conditions — where threads overwrite each other's data.

Q7. What is Synchronization in Multithreading?

Synchronization is a mechanism that controls access to shared resources to prevent data inconsistency by allowing only one thread to access a resource at a time.

Q8. What are the different methods of threads?

Common thread methods:

- `start()` – starts a new thread
- `run()` – defines the thread's task
- `sleep()` – pauses thread temporarily
- `join()` – waits for another thread to finish
- `getName()`, `setName()` – manage thread names
- `getPriority()`, `setPriority()` – manage priorities

Q9. How to create a thread? How many ways to create a thread?

There are two ways to create a thread in Java:

1. By extending Thread class and overriding `run()`
2. By implementing Runnable interface and passing it to a Thread object.

Q10. Difference between `join()`, `wait()`, and `sleep()`:

Method	Class	Use
<code>join()</code>	Thread	Waits for a thread to finish
<code>wait()</code>	Object	Waits until notified
<code>sleep()</code>	Thread	Pauses for some time

Q11. Difference between notify() and notifyAll():

Method	Use
notify()	Wakes up one waiting thread
notifyAll()	Wakes up all waiting threads

Q12. Explain Thread Life Cycle:

1. New – Thread is created
2. Runnable – Ready to run
3. Running – Actively executing
4. Waiting/Blocked – Temporarily inactive
5. Terminated/Dead – Execution complete

Q13. What is MAX and MIN priority of the thread?

- MAX_PRIORITY = 10
- MIN_PRIORITY = 1
- NORM_PRIORITY = 5

Q14. Can we control thread?

Yes, by using methods like sleep(), yield(), join(), and setPriority() we can control thread execution.

15. What is a Functional Interface?

A Functional Interface has exactly one abstract method and can be used with lambda expressions.

Example: Runnable, Callable, Comparable.